Dependency Injection

Dependency is an object that other object needs to perform its operation.

For example,

```
Car
 ↓
Engine
```

Here, the **Car** depends on the **Engine**.

In traditional Java, the class creates its own dependency using the `new` keyword.

```
Engine engine = new Engine();
```

This creates **tight coupling** because the `Car` class is directly responsible for creating the `Engine`.

Spring solves this problem using **Dependency Injection (DI)**.

Definition

> **Dependency Injection (DI)** is a design pattern in which the Spring IoC Container creates the required objects (dependencies) and injects them into other objects automatically instead of allowing the objects to create their own dependencies.

Benefits of DI

- Loose coupling
- Easier testing
- Better code reusability
- Highly Scalable
- More flexible

Types of Dependency Injection

Spring supports three types:

1. Constructor Injection (Recommended)
2. Setter Injection
3. Field Injection (`@Autowired`)

2. **Constructor Injection**

   Dependencies are provided through the **constructor** when object is created.

   Use Constructor Injection when the dependency is **mandatory**.

ExampleEngine Class

```
@Component
public class Engine {
}
```

Car Class

```
@Component
public class Car {

    private Engine engine;

    @Autowired
    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

`@Autowired` tells Spring to look up the matching bean in its context and plug it in for you.

Flow:

```
Spring Container
      |
Creates Engine
      |
Creates Car
      |
Calls Constructor
      |
Injects Engine
```

Advantages

- Mandatory dependencies
- Immutable objects
- Easy to test
- Recommended by Spring

3. **Setter Injection**

   In Setter Injection, dependencies are injected using setter methods after the object has been created.

   Use Setter Injection when the dependency is **optional**.

Example

```java
@Component
public class Car {

    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) {
        this.engine = engine;
```

```
    }
}
```

Flow:

```
Create Car
     |
Call setEngine()
     |
Inject Engine
```

Advantages

- Optional dependencies
- Dependencies can be changed later

4. **Fielder Injection**

   The dependency is injected directly into the field.

Example

```java
@Component
public class Car {

    @Autowired
    private Engine engine;
}
```

Explanation

Spring uses reflection to inject the `Engine` bean directly into the field.

Although simple, **Field Injection is generally not recommended** because:

- **Difficult to test**

    - It is difficult because, in unit testing we should able initialize the class, pass in a mock object and test its methods in isolation.
    - **With Constructor Injection:** You simply call `MyService service = new MyService(mockRepo);`
    - **With Field Injection:** Doing `MyService service = new MyService();` leaves all of your private dependencies initialized as `null`. Since there is no public constructor or setter exposing those dependencies, you cannot pass your mocks into the object directly.
- Hidden dependencies
- Cannot create immutable classes

Constructor vs Setter Injection

Constructor Injection	Setter Injection
Mandatory dependency	Optional dependency
Uses constructor	Uses setter method
Immutable	Mutable
Recommended	Used when required

5. **Autowiring dependencies**

   Normally we tell Spring exactly which dependency to inject.

   With **Autowiring**, Spring automatically finds the required bean and injects it.

   ## Example

   ```
   @Component
   public class Car {

       @Autowired
       private Engine engine;
   }
   ```

Explanation

Spring searches for a bean of type `Engine`.

If found:

```
Spring Container
      |
Find Engine Bean
      |
Inject into Car
```

No manual object creation is required.

How Autowiring Works

Spring primarily resolves dependencies:

1. By Type (most common)
2. By Name (if needed)
3. Using Qualifiers (when multiple beans exist)

6. **Autowiring Conflict Problem**

   Suppose there are two Engine beans.

```
@Component
public class PetrolEngine {
}
```

```
@Component
public class DieselEngine {
}
```

Now:

```
@Autowired
private Engine engine;
```

Spring doesn't know which bean to inject.

Result:

```
NoUniqueBeanDefinitionException
```

7. **Qualifier**

   `@Qualifier` tells Spring exactly which bean should be injected.

   ## Example

   ```
   @Component("petrolEngine")
   public class PetrolEngine implements Engine {
   }
   ```

   ```
   @Component("dieselEngine")
   public class DieselEngine implements Engine {
   }
   ```

   Inject the required bean:

   ```
   @Autowired
   @Qualifier("petrolEngine")
   private Engine engine;
   ```

   Spring now injects only the **petrolEngine** bean.

8. **Resource Annotation**

   `@Resource` is a Java EE (Jakarta) annotation used for dependency injection.

   it inject based on name.

Example

```
@Resource
private Engine engine;
```

If a bean named `engine` exists, Spring injects it.

---

9. **Inject Annotation**

   `@Inject` is part of **Jakarta Dependency Injection (CDI)**.

   It behaves similarly to `@Autowired`.

Example

```
@Inject
private Engine engine;
```

Spring injects the dependency automatically.

If multiple beans exist:

```
@Inject
@Named("petrolEngine")
private Engine engine;
```

@Autowired vs @Resource vs @Inject

Annotation	Package	Injection Method
`@Autowired`	Spring	By Type (default)
`@Resource`	Jakarta	By Name (default)
`@Inject`	Jakarta CDI	By Type (similar to `@Autowired`)

When to Use Which?

Situation	Recommended
Spring applications	`@Autowired`
Standard Java/Jakarta EE	`@Inject`
Need injection by bean name	`@Resource`
Multiple beans of same type	`@Qualifier` with `@Autowired`